

SEQUENTIAL SYSTEMS

Phase IV: Synchronous Orchestration

Comprehensive Academic Master Manual

Optimized Visual Layout Edition:

Scaled Diagrams & Expanded Component Spacing

Strict Anti-Overlap Formatting

Methodology: Pure $\text{\LaTeX}$ Compilation

EE/PC/B/T/225 | Jadavpur University

Contents

1	The Synchronous Paradigm	2
1.1	Solving the Propagation Crisis	2
1.1.1	The Parallel Clock Topology	2
1.2	The Trade-off: Hardware Complexity	4
2	Synthesis: Mod-5 Synchronous Counter	5
2.1	Algorithmic FSM Design	5
2.1.1	Step 1: State Allocation	5
2.2	Step 2: The State Synthesis Table	6
2.3	Step 3: Karnaugh Map Minimization	8
2.3.1	K-Map for D_0 (LSB)	8
2.3.2	K-Map for D_1	9
2.3.3	K-Map for D_2 (MSB)	10
2.4	Step 4: Logical Hardware Implementation	10
3	Synthesis: Mod-4 Synchronous Up-Counter	12
3.1	Algorithmic T-Flip-Flop Design	12
3.1.1	Step 1: State Allocation	12
3.1.2	Step 2: The State Synthesis Table	13

3.1.3	Step 3: Logic Extraction	13
3.2	Step 4: Timing Diagram Proof	14

Chapter 1

The Synchronous Paradigm

1.1 Solving the Propagation Crisis

In Phase III, we explored the inherent limitation of Asynchronous (Ripple) Counters. Because the clock signal must physically cascade from the LSB flip-flop through to the MSB flip-flop, the state transitions are delayed. The total settling time is the cumulative sum of all individual propagation delays ($N \times t_{pd}$).

This delay creates dangerous decoding glitches and severely restricts the maximum operating frequency of the circuit.

1.1.1 The Parallel Clock Topology

To eliminate these timing hazards, engineers developed **Synchronous Counters**.

In a synchronous architecture, the master clock signal from the oscillator is wired **simultaneously** (in parallel) to the clock pins of every single flip-flop in the

array.

Because all memory elements receive the triggering edge at the exact same picosecond, they all evaluate their inputs and update their outputs in absolute unison. The total settling time of a synchronous counter is equal to the propagation delay of just *one* flip-flop, regardless of how many bits the counter possesses.

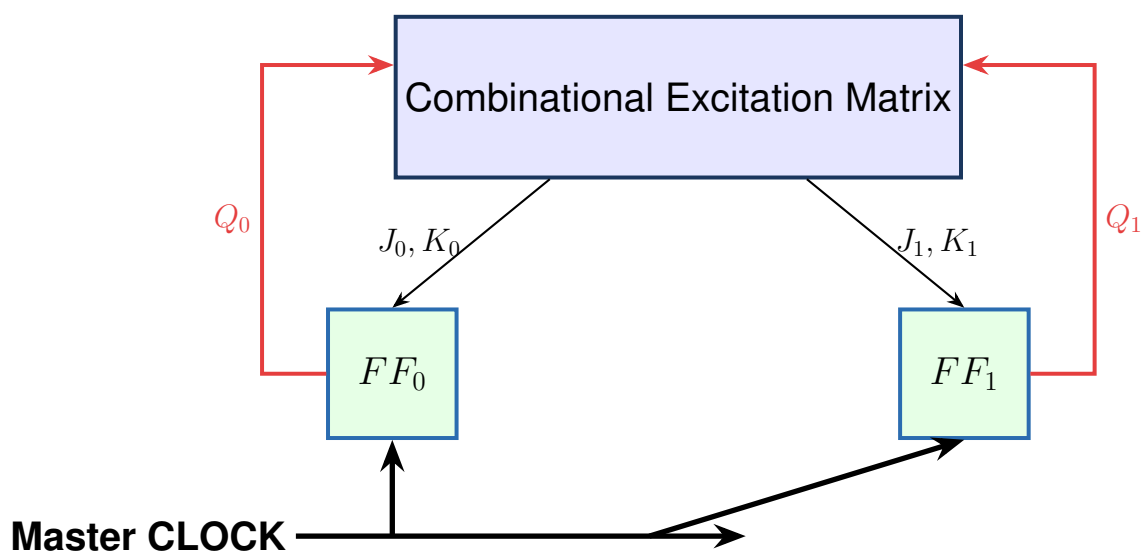
1.2 The Trade-off: Hardware Complexity

In a ripple counter, the sequencing logic is derived simply by wiring Q to the next clock.

However, in a synchronous counter, because all flip-flops are clocked simultaneously, we can no longer rely on the clock cascade to generate the binary sequence. We must explicitly calculate what the next state of each individual flip-flop should be, and design a **Combinational Logic Matrix** to feed the precise J , K , D , or T voltages into the input pins before the clock edge arrives.

The Fundamental Synchronous Rule

A synchronous counter is essentially a specific type of Finite State Machine (FSM). We must use Excitation Tables and Karnaugh Maps to derive the Boolean equations for every single input pin of every single flip-flop.



Chapter 2

Synthesis: Mod-5 Synchronous Counter

2.1 Algorithmic FSM Design

Let us execute the strict STC Sir methodology to design a Synchronous Mod-5 Counter using **D Flip-Flops**.

2.1.1 Step 1: State Allocation

The objective is to design a counter that counts: 0, 1, 2, 3, 4, and then resets back to 0. It must possess exactly 5 distinct states.

Because $2^2 < 5 \leq 2^3$, we mathematically require **3 Flip-Flops** (Q_2 MSB, Q_1 , Q_0 LSB). The sequence will utilize states 000 through 100. States 101 (5), 110 (6), and 111 (7) will not be used in the normal counting sequence.

2.2 Step 2: The State Synthesis Table

We must define the present state, the target next state, and consult the D-Flip-Flop excitation table.

D-Flip-Flop Excitation Reminder

The D Flip-Flop is uniquely transparent. The required input voltage D is always exactly equal to the desired Next State ($D = Q_{n+1}$). Therefore, $D_2 = Q_{2(next)}$, $D_1 = Q_{1(next)}$, and $D_0 = Q_{0(next)}$.

The "Don't Cares" (X) for states 5, 6, and 7 will be critical for logic minimization in the next step.

Present State			Next State			Required Excitation		
Q_2	Q_1	Q_0	Q'_2	Q'_1	Q'_0	D_2	D_1	D_0
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	0
0	1	0	0	1	1	0	1	1
0	1	1	1	0	0	1	0	0
1	0	0	0	0	0	0	0	0
1	0	1	X	X	X	X	X	X
1	1	0	X	X	X	X	X	X
1	1	1	X	X	X	X	X	X

2.3 Step 3: Karnaugh Map Minimization

We construct three 3-variable K-Maps to derive the combinational logic for the D_2 , D_1 , and D_0 pins.

2.3.1 K-Map for D_0 (LSB)

	Q_1Q_0			
Q_2	00	01	11	10
0	1	0	0	1
1	0	X	X	X

Minimization Logic: We group the 1 at minterm 0 with the 1 at minterm 2 (wrapping around the edges). Q_2 must be 0, and Q_0 must be 0.

Final Equation: $D_0 = \overline{Q_2} \cdot \overline{Q_0}$

2.3.2 K-Map for D_1

	$Q_1 Q_0$			
Q_2	00	01	11	10
0	0	1	0	1
1	0	X	X	X

Minimization Logic: We group the 1 at minterm 2 with the X at minterm 6. We isolate the 1 at minterm 1 (cannot be grouped).

$$D_1 = Q_1 \overline{Q_0} + \overline{Q_2} \overline{Q_1} Q_0.$$

Since state 101 (minterm 5) is a don't care, we can assume it is 1 to group with minterm 1, yielding $\overline{Q_1} Q_0$. Thus:

Final Equation: $D_1 = Q_1 \oplus Q_0$ (Standard binary counter behavior for the 2nd bit).

2.3.3 K-Map for D_2 (MSB)

	$Q_1 Q_0$			
Q_2	00	01	11	10
0	0	0	1	0
1	0	X	X	X

Minimization Logic: We group the 1 at minterm 3 with the X at minterm 7. Q_2 drops out.

Final Equation: $D_2 = Q_1 \cdot Q_0$

2.4 Step 4: Logical Hardware Implementation

Based on our rigorous mathematical derivation, we assemble the final circuit.

- $FF_0 (D_0)$ receives the output of an AND gate computing $\overline{Q_2} \cdot \overline{Q_0}$.

- $FF_1 (D_1)$ receives the output of an XOR gate computing $Q_1 \oplus Q_0$.
- $FF_2 (D_2)$ receives the output of an AND gate computing $Q_1 \cdot Q_0$.
- **Crucial Feature:** All three clock pins are physically shorted to a single master clock wire.

Because the excitation logic prepares the exact voltages before the clock edge arrives, the entire 3-bit array will update simultaneously to the precise next state (0, 1, 2, 3, 4, 0...) upon every active clock pulse. No glitches will occur.

Chapter 3

Synthesis: Mod-4 Synchronous Up-Counter

3.1 Algorithmic T-Flip-Flop Design

Let us reinforce the synchronous methodology by designing a standard 2-bit (Mod-4) Up-Counter using **T Flip-Flops**.

3.1.1 Step 1: State Allocation

The objective is to design a counter that counts: 00, 01, 10, 11, and then overflows to 00. It possesses exactly 4 states. We require **2 T Flip-Flops** (Q_1 MSB, Q_0 LSB).

3.1.2 Step 2: The State Synthesis Table

Consult the T-Flip-Flop excitation table. $T = 1$ is required to change state. $T = 0$ is required to hold state.

Present State		Next State		Required Excitation	
Q_1	Q_0	Q'_1	Q'_0	T_1	T_0
0	0	0	1	0	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	0	0	1	1

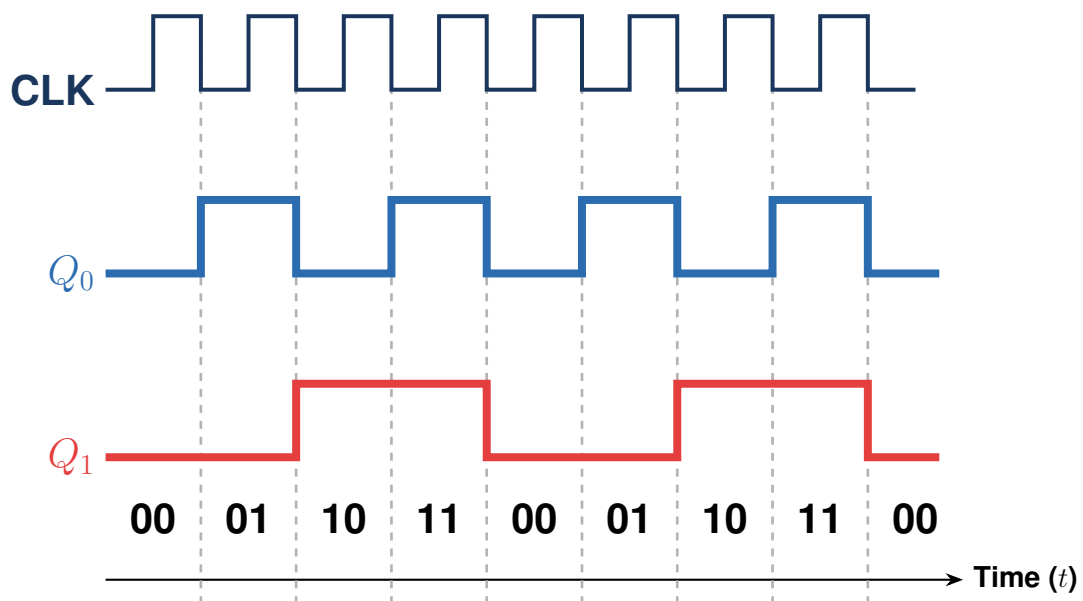
3.1.3 Step 3: Logic Extraction

By visual inspection of the excitation columns, the logic equations resolve beautifully without the need for complex K-Maps:

- $T_0 = 1$: The LSB toggles on every single clock pulse.
- $T_1 = Q_0$: The MSB only toggles when the LSB is currently saturated HIGH (1).

3.2 Step 4: Timing Diagram Proof

The hallmark of a synchronous counter is perfectly aligned state transitions. Let us construct the timing diagram for the derived Mod-4 T-Flip-Flop architecture over 6 clock pulses, assuming Negative-Edge Triggering.



Visualizing Synchronization

Observe the absolute vertical alignment at $x = 4$ and $x = 8$. When Q_0 and Q_1 must update simultaneously, their voltage transitions occur on the exact same vertical dashed line (the clock edge). There is no "staircase" ripple effect, and therefore, no decoding glitches.

***** END OF PHASE IV MASTER
MANUAL *****